

36
Дек 10

useRef: Persistência e Manipulação de Valores no React

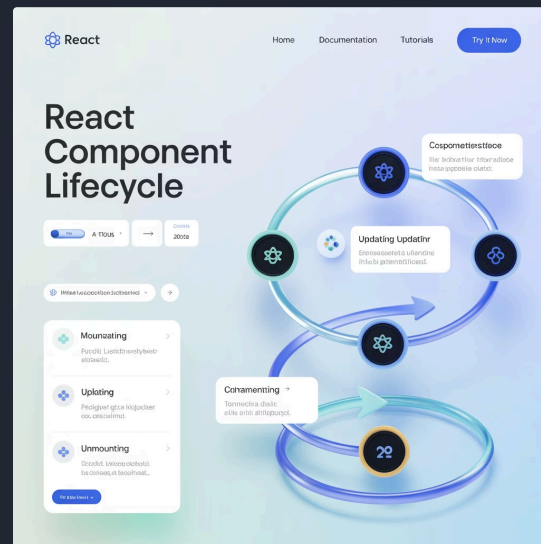
Bem-vindo a esta aula sobre o hook useRef no React! Vamos explorar como esta ferramenta poderosa permite manter valores persistentes entre renderizações sem causar atualizações desnecessárias na interface.

O que é o useRef e qual seu propósito?

No React, **cada renderização reinicia o escopo da função** do componente. Todas as variáveis são redefinidas a cada atualização.

O useRef oferece um objeto mutável que **persiste entre renderizações**, permitindo:

- Criar referências a elementos do DOM
- Armazenar valores que não devem causar re-renderizações



Ao contrário de variáveis comuns, o valor dentro de useRef permanece intacto entre atualizações de componente, tornando-o ideal para persistência de dados.

Estrutura e Funcionamento do useRef



Sintaxe Básica

```
const meuRef = useRef(valorInicial);
```

O hook aceita um valor inicial e retorna um objeto mutável que persiste entre renderizações.



Estrutura do Objeto

```
{ current: valorAtual }
```

O valor é sempre acessado e modificado através da propriedade **current**, que pode ser alterada diretamente.



Identidade Imutável

O objeto retornado pelo useRef permanece o **mesmo entre renderizações**, apenas o valor de **.current** pode mudar.

useRef vs useState: Entendendo as Diferenças

Característica	useRef	useState
Persiste entre renderizações	✓	✓
Causa re-renderização	✗	✓
Acesso ao DOM	✓	✗
Ideal para lógica de UI	✗	✓

Use **useRef** quando precisar armazenar valores que não afetam diretamente a renderização ou quando necessitar manipular elementos DOM.

Use **useState** quando o valor precisa controlar a interface do usuário e refletir visualmente mudanças de estado.

Casos de Uso Comuns do useRef

Manipulação do DOM

- Focar inputs automaticamente
- Selecionar texto em campos
- Medir dimensões de elementos

Persistência de Valores

- Contadores internos
- Estado anterior de props/state
- Flags de controle de fluxo

Referências a Timers

- IDs de setTimeout/setInterval
- Limpeza de efeitos assíncronos
- Controle de animações



Exemplo Prático de useRef

```
import React, { useRef, useEffect } from 'react';

function AutoFocus() {
  const inputRef = useRef(null);
  const renderCount = useRef(0);

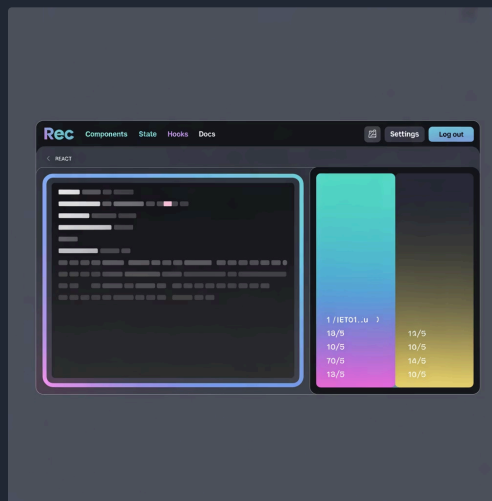
  useEffect(() => {
    // Foca o input ao montar
    inputRef.current.focus();

    // Conta renderizações sem causar loop
    renderCount.current += 1;
    console.log(`Renderizou ${renderCount.current} vezes`);
  }, []);

  return (); }
```

Pontos importantes:

- O **inputRef** acessa diretamente o elemento DOM
- O **renderCount** mantém um contador que não causa re-renderizações
- Alterar **.current** não dispara atualizações na UI
- Os valores persistem mesmo quando o componente re-renderiza



Cuidados e Melhores Práticas



Não use para estado visual

Se um valor precisa refletir mudanças na UI, use `useState`. O `useRef` não dispara re-renderizações quando seu valor muda.



Atualizações assíncronas

Lembre-se que mudar `useRef.current` não causa efeito imediato. Use-o em conjunto com `useEffect` para sincronizar quando necessário.



Evite acoplamentos complexos

Mantenha a manipulação do DOM via `useRef` simples e direta. Separe as responsabilidades entre lógica e visualização.

O `useRef` é uma ferramenta poderosa para otimizar a performance e simplificar a lógica do seu componente quando usado corretamente.